

## Problem Statement

Write a C, Java, or Python program to determine the shortest path, in terms of total driving distance, between two locations in a road network represented as a weighted undirected graph. The weight of each edge is at least as large as the Euclidean distance between the corresponding vertices. The road network data is provided in a text file named `graph_data.txt`. Each node in the graph represents a location with specific coordinates, and each edge represents a road with a given driving distance. Your program must read this file and implement a search algorithm to find the optimal path from a specified starting location to a target location. You must implement the search algorithm yourself; do not use built-in library functions or modules (e.g., `networkx` in Python) that directly compute shortest paths. You cannot use Dijkstra's algorithm.

## Input

An example of the content of the text file named `graph_data.txt` is as follows:

```
5 6
96.695882 30.864883
48.188706 27.740254
40.391313 98.568165
5.362280 8.607419
19.946435 79.894819
0 1 57.799213
1 2 73.386690
1 3 72.435762
1 4 66.158788
2 4 42.195245
3 4 85.065436
```

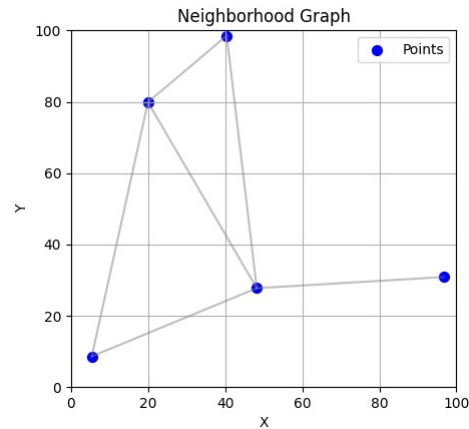
The first and second entries are integers  $n$  and  $e$ , indicating the number of nodes and the number of edges. In this example,  $n$  is 5 and  $e$  is 6. The next  $n$  lines are in

$x \ y$

format, where  $x$  and  $y$  indicate each vertex's first and second coordinates. The last  $e$  lines are in the

$u \ v \ w$

format, where  $u$ ,  $v$ , and  $w$  indicate the existence of an undirected weighted edge between nodes  $u$  and  $v$  with weight  $w$ . This example represents the following graph:



## Output

Assuming the user wants the path from node 2 to node 3 following is the output:

```
Enter the start node index (0-4): 2
Enter the goal node index (0-4): 3
Path (from start to goal):
2 -> 4 -> 3
Total Path Cost: 127.26
```

## Submission Guidelines

Submit your source code as `route_<your roll>.x`, where  $x$  is either `c`, `java`, or `py`, and a file `output_<your roll>.txt` containing the output in the above format, which shows the optimal path and its cost.

## Marks Distribution

Marks will be awarded as follows (Total 100): 45 marks for correct search algorithm implementation; 20 marks for proper input file handling and graph representation; 25 marks for accurate shortest path and cost calculation; 5 marks for correct output formatting; and 5 marks for code quality (readability, comments, variable names).